

ROLL NO: 240701382

NAME: M.PERIYA NAYAGAM

1. Comparative Study of Low-Code Development versus Traditional Application Development Approaches

1. Aim

To carry out a comparative examination of low-code development and conventional application development approaches, analyzing their respective phases, workflows, and technical distinctions.

2. Introduction to Traditional Approach

The conventional method of building software depends on writing code manually, line by line, using standard code editors and integrated development environments (IDEs). Developers are responsible for manually configuring database structures, connecting APIs, and handling server deployment. Because this process demands strong technical proficiency in languages such as Java, Python, or C++, it is highly dependent on skilled personnel. Although this method provides complete flexibility and full command over the codebase and system architecture, it generally results in longer project timelines, higher budget requirements, and thorough testing for defects.

3. Introduction to Low-Code Approach

Low-Code Application Platforms (LCAPs) substitute heavy manual coding with visual, cloud-hosted development tools. These platforms make use of drag-and-drop components, ready-made visual templates, and automated deployment processes. This approach simplifies the learning curve, enabling “citizen developers” with little or no programming background to create working applications quickly. Through automation of routine coding work, low-code significantly reduces time-to-market and initial development expenses, though it can restrict deep customization and create dependency on a particular platform.

4. Diagram for Stages in Traditional Approach

The conventional Software Development Life Cycle (SDLC) follows a linear or iterative sequence that requires dedicated engineering effort at every stage:



6. Comparative Analysis (Tabulation)

Feature	Traditional Development	Low-Code Development
Development Speed	Slower, since every component is coded manually from the ground up.	Quicker — cuts development time by nearly 50-70% through ready-made modules.
Skill Required	High; needs experienced programmers and engineers.	Low; allows citizen developers to build using drag-and-drop tools.
Customization	Unlimited; developers have full control over the entire codebase.	Restricted; limited to whatever features the platform provides.
Cost	Higher initial investment, staffing costs, and ongoing maintenance expenses.	Reduced upfront expense, though subject to recurring license or subscription fees.
Maintenance & Updates	Time-consuming, as every fix and update needs manual coding and testing.	Streamlined; updates are managed and rolled out centrally by the platform provider.
Vendor Lock-in	Absent; the business retains complete ownership of its source code.	Significant; the application relies on the platform provider's ecosystem.
Best Use Cases	Ideal for complex, high-performance, and highly secure enterprise systems.	Suited for prototyping, workflow automation, and internal-use tools.

7. Result

This comparative study shows that while traditional development ensures maximum flexibility, precise control, and tailored scalability suited for mission-critical enterprise systems, low-code development offers a fast, economical, and agile method for building, launching, and maintaining business applications. Many organizations find it advantageous to adopt a hybrid model — using low-code for rapid front-end and workflow development while relying on traditional coding for intricate, custom back-end requirements.